CanSat XBee Comms Library — Design Document

Harre Prakash Kumar · TMU CanSat 2026–27 Flight Software Onboarding

The library is split into two independent layers. The **XBee layer** builds 0x10 Transmit Request frames, parses 0x90 Receive Packet frames, and assembles frames from a UART byte stream; it treats the RF data as opaque bytes. The **payload layer** converts the three message structs to and from bytes and knows nothing about addresses, frame IDs or checksums. Each layer is tested on its own and then together.

1. Designing for a microcontroller without dynamic allocation

- **Caller-owned, fixed-size buffers.** Every size is a compile-time constant (`kMaxTransmitFrameSize` = 274 B, `kMaxPayloadSize` = 7 B), so buffers can be static or on the stack and worst-case RAM is known at build time. The library itself contains no `new`, `malloc`, STL containers or exceptions (verified by inspecting the library's undefined symbols); it also compiles with `-fno-exceptions -fno-rtti`.
- **Zero-copy receive.** `parse_receive_packet` returns a `ByteView` (pointer + length) into the received frame instead of copying the payload. Transmit performs exactly one copy, of the payload into the outgoing frame, which is unavoidable.
- **Streaming input.** `FrameReader` accepts one byte at a time, as a UART interrupt or DMA ring buffer delivers it, using a single 272-byte internal buffer (296 B object). It can be driven directly from an ISR or a polling loop.
- **Status codes instead of exceptions.** Every operation returns a small `Status` enum, which is deterministic, costs no unwinding tables, and tells the caller exactly why input was rejected.

Trade-offs: the receive buffer is sized for the worst-case 256-byte RF payload even though our largest message is 7 bytes; this wastes ~250 B of RAM but means the reader never has to reject a legitimate frame from another source. Zero-copy views require the caller not to overwrite the frame buffer while the payload is in use. C++17 is used (with a minimal `ByteView` instead of `std::span`) because it is supported by every common Arm embedded toolchain.

2. Choosing the application payload format

ID	Message	Size	Layout
0x01	OutTelemetry	7 B	`[0x01][seconds_since_epoch u32 BE][mechanisms_deployed_flags u16 BE]`
0x02	CommandMechanism	3 B	`[0x02][mechanism_id u8][value u8]`
0x03	CommandTelemetry	2 B	`[0x03][is_on u8: 0x00 off / 0x01 on]`

- **Message identification:** a 1-byte type ID at the start of every payload lets the receiver dispatch with a single lookup, leaves room for 252 future message types, and makes a misrouted message (e.g. telemetry arriving on the uplink) detectable. ID 0x00 is deliberately unused so that an all-zero payload is never valid.
- **Field sizes:** each field keeps the width of its struct member, so no information is lost and no bytes are wasted. `bool` is encoded as one explicit byte because its in-memory size and representation are implementation-defined. There is no padding, so telemetry is 7 bytes versus 8 for `sizeof(OutTelemetry)`.
- **Byte order:** big-endian (network order), matching the XBee's own length and address fields. That makes captured frames easy to read and keeps the format independent of the CPU (Cortex-M is little-endian, as is the Ground Station PC). Fields are packed with shifts, never by `memcpy` of a struct, which avoids padding, alignment and endianness bugs.
- **No length or CRC inside the payload:** the XBee frame already carries a length and checksum, and every message has a fixed size determined by its ID, so repeating them would only add overhead. A version byte was considered but left out for simplicity; unused type IDs can be used to introduce new message versions later.

3. Handling malformed, incomplete or invalid input

Validation happens in layers, each rejecting only what it understands:

- **Frame layer:** the start delimiter must be 0x7E; the length field must exactly match the bytes provided (too few gives *Incomplete*, extra gives *LengthMismatch*); the checksum must validate; the type must be 0x90 and the frame must be long enough for its 12-byte header (*MalformedFrame*). Every bounds check happens before the byte is read.
- **Stream layer:** noise before 0x7E is skipped; a length larger than the buffer is rejected as soon as the length bytes arrive, so it can never overflow; after any error the reader discards the frame, increments an error counter, and re-synchronises at the next 0x7E. One corrupt frame therefore never affects the following ones.
- **Payload layer:** unknown IDs, a valid ID in the wrong direction, payloads shorter or longer than their fixed size, and booleans other than 0x00/0x01 are all rejected. Strictness is intentional: on a flight vehicle, guessing at a corrupt command is worse than ignoring it.

What the Flight Software should do: drop the bad frame and keep running. It should never crash, reset or act on partial data. It should increment a per-reason error counter and include it in telemetry so the Ground Station can see link quality, keep its current state (for example, a mechanism is not actuated on a bad command), and rely on the Ground Station to resend, which it can confirm when the resulting state appears in telemetry. On transmit, errors such as *BufferTooSmall* indicate programming bugs and should be caught in testing. The 60-test Google Test suite checks every single-bit flip and every truncation of a frame and passes under AddressSanitizer and UBSan.